

ISYE-6740-Final-Project: Classification for Piloting Reactive Images

Team Members: Jonathan Feng

Spring 2025

Background

In the world of social media and online activities, anonymity is nothing new. Even when it isn't a requirement, some people still choose to represent themselves through means . It's hard to believe there aren't days where people don't want to turn on their webcams . What better solution is there than to hide behind a reactive image or a picture perfect virtual model. The technology behind virtual avatars have grown drastically throughout the past few years, think of Apple, Meta-verse, and V-tubers. I believe the technology is here to stay and will keep on gaining traction.

However, do not take these models lightly, they can be quite costly and have lots of overhead. Starting from the most complex to the least come 3D-avatars. The technology can financially set you back anywhere from 500 - 600 dollars, to multiple thousands, this is not including some of the other assets needed. 2D-avatars are no different, requiring a good enough phone to track your movements and the upfront cost of models with a similar range .Custom art pieces with rigging come at drastically different price ranges but it is usually a "you get what you pay for" scenario.

The least costly option is a reactive png image. Think of a profile picture that lights up during zoom calls, it's quite similar but more flexible since the art and effort needed to create one is, on average, less. The only drawback of the last option is the lack of range in facial expressions, it is only limited by how much you want to spend.

Problem Statement

Given the cheapest price point and simplest solution, static images are the way to go if you don't care to invest more. Let's just say a creator decides to go with

the cheapest option. Great! What now? Now you have to toggle your emotions manually as you talk if you chose to have more than one facial emotions. This is the exact issue I wanted to tackle. To combat having to manually toggle these expressions, we will be attempting to build a model to predict what emotion will be conveyed so we can automate this process.

We will be exploring a variety of traditional machine learning models and see how it stacks up against some of the more modern practices like adapting on top of an language model.



Figure 1: Example of what is possible ft. artist cikanindya

Data Wrangling

The data will be primarily focused on chat messages from, Twitch.tv, a popular streaming platform. The idea behind my process is that it would be efficient collecting messages with associated emotes. More often than not, if you are using an emote with a message it is to emphasize the meaning behind it. In our terms, the emotes will make our "labeling of emotions", easier and be a proxy for labeling our emotions.

I wrote a script to collect raw chat data via Twitch's IRC (internet relay chat) servers, and made it reusable so that you can run multiple at a time. Monitoring different channels at the same time using python and bash was a blessing. This process was performed over the course of 4 weeks, 3-4 days a week, and around 10-15 channels a day. Channel selection was also carefully considered as some communities may be more prone to having data we are not looking for, more on that later.

```

> ls
__pycache__      chatlogs-old      distilbert-emote-lora      final_results_text
chat_eda.py       cleaning_r1.py     distilbert-emote-refine-model modeling.py
chat_scraper.py   cleaning_r2.py     distilbert-emote-refine-token official_dataset
chatlogs          distil_data.py     fasttext                   other_dataset_version

> python chat_scraper.py --channel Gigguk

```

Figure 2: Running the twitch scraper

```

Emotes:[]
written
['Without pee?', '[]', '[]', 'keju_manis']
Received ---
@badge-info=;badges=;color=#2E8B57;display-names=Daedalusmk7;emotes=;first-msg=0;flags=;id=67d6508a-623c-48ff-8c80-5d7e0c407bee;mod=0;returning-chatter=0;room-id=24411833;subscriber=0;tmi-sent-ts=1745558610751;turbo=0;user-id=57529236;user-type=:daedalusmk7!daedalusmk7@daedalusmk7.tmi.twitch.tv PRIVMSG #gigguk :snipe Connor's stream while you eat.
Daedalusmk7: snipe Connor's stream while you eat.
Emotes:[]
written
['snipe Connor's stream while you eat.', '[]', '[]', 'Daedalusmk7']

```

Figure 3: Scraper example logs

Thinking I had struck gold with the large influx of chat messages coming in, I quickly ran into issues. There was a lot more to text processing than I previously thought. The variety of cases to account for and not limited to were: Chat bot messages, third party emotes, obscure community culture, inconsistent character encoding.

Aside from the base emotes on the Twitch platform, there has been a rise of using third party emotes through extensions like 7TV, BBTv, and FrankerZ Face, to circumvent the need to subscribe to use fun emotes. Due to the sheer number and lack of moderation in these type of environments, there are probably tens of thousands of these custom emotes floating around out there. I made the executive decision to grab the 100 most commonly used emotes from each of those platforms, totaling 300 in tandem with all current approved twitch emotes.

After painstakingly accounting for most of these edge cases, I finally won the battle right? Oh how wrong I was. Most of the data collected was unusable due to the simple nature of people either responding with spam messages, text only, or emoji, nullifying all the efforts I put into careful channel selection. People are quite unpredictable! When I was finished culling all the obvious anomalies, only a bit over 15,000 messages remained. With that said, the next upcoming sections will cover the EDA and data transformations made to this raw data.

Data Source

The final data I am using consists of the columns message, emotes, username, weights, and final emotion. Below is a short tabled reference of what the final schema looks like before I start the modeling process.

Feature	Data Type	Description
message	str	The message the user sent without the emotes
emotes	array<str>	An array of emotes used in their text representation
username	str	The user that sent the message
final_weights	dict<str,float>	These are the corresponding weights from zero shot bart-large-mnli and all-MiniLM-L6-v2 for cosine similarity with a final override weight for obvious emotes.
final_emotion	str	Possible categories: ['anger', 'disgust', 'fear', 'happiness', 'sadness', 'surprise', 'neutral'] from Paul Ekman's 6 basic emotions + Neutral

Table 1: Table Variable References

The message, emotes, username, and final_emotion are described within the table. For final_weights, I would like to dive deeper into how this is calculated. Due to the nature of how much data there is, labeling completely by hand is virtually impossible. So, we will need to do things in a slightly smarter way:

1. Using all-MiniLM-L6-v2 to predict the emotion category calculate a weight between 0-1 for each emotion based on cosine similarity for each emote in emote array.
2. Using a zero shot method with bart-large-mnli to calculate similarity between 0-1 for each emote in emote array.
3. Having an overwhelming weight applied for popular known emotes that have little variation in interpretation.
4. Summing these all up and taking the maximum as our labeled emotion.
5. Hand checking results to assess accuracy.

I did try a variety of auto labeling methods including usage of spacy and other

popular lightweight models. However, the two mentioned above worked out the best for my use case. After doing some additional filtering for spam messages, exploring initial insights through common data exploration methods, and unaccounted for errors, we arrive at a dataset that looks something like this:

message	emotes	username	final_weights	final_emotion
why does my message keep getting buried in chat	['ctvaCry', 'ctvaCry', 'ctvaCry']	Kjsgamer	{... 'sadness': 0.2...}	sadness
girl are you dying rn	['ctvaWheeze']	snow011011	{... 'hapiness': 0.2...}	happiness
i knew it	['ICANT']	NinjaCaribou	{... 'surprise': 0.2...}	surprise
unless you mod it for more	['ok']	Veekha	{... 'neutral': 0.2...}	neutral
all it takes is one time	['YEP']	TakaBakashi	{... 'neutral': 0.2...}	neutral

Table 2: Tabled fields of finalized data from scraping

Now that we have usable data in the form we want, we can assess our data and check for quality in the true data exploration phase.

Data Exploration

In this phase of the project, I will be accounting for outliers and exploring initial insights through common exploratory data analysis methods. Due to the nature of casual chat messages being so unpredictable, we will also delve into why certain aspects of the distribution are behaving a certain way, and what I did to address it.

We begin with 15,258 rows of data across 4,748 unique users. Below are the initial distribution across all 7 emotions followed by the most common emotes per class. The emote weights heat map is also shown to assess my labeling. Note that the labels may seem skewed because some messages take into account multiple emotes that may be closer to one emotion than the other by a hair.

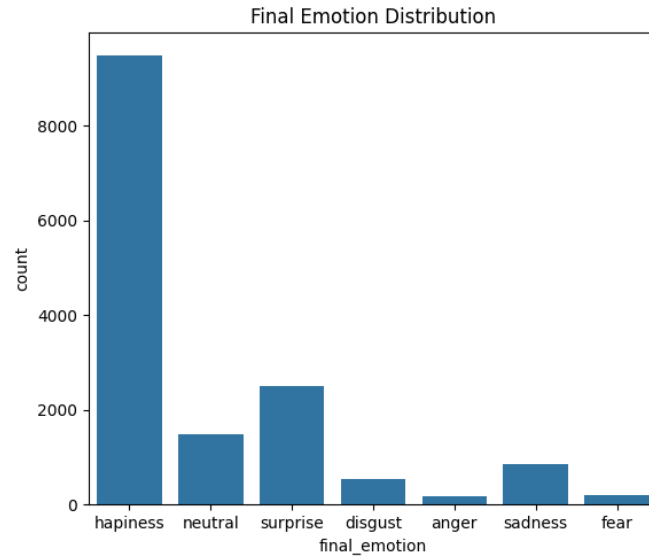


Figure 4: Emotional distribution across all 7 emote labels.

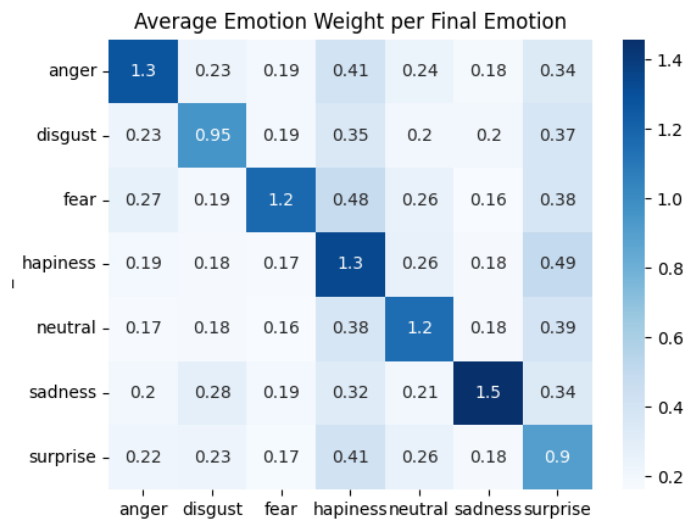


Figure 5: Average emotion per weight across all 7 emote labels

As you might notice already the distribution of the emotes is highly skewed. My

main interpretation for this is that in online communities and highly moderated ones like Twitch, are going to be generally positive. You won't be seeing many emotions like anger, sadness, and disgust since people are just trying to have a good time. If there are any instances, many would be akin to reactions, one-liners, or single emojis representing an instance of the minority emotions. Those will not show up in our data because they don't have the necessary data we are looking for labeling at all.

Given this, we cannot consider those minority emotions. The heatmap mainly serves as a check if any two emotion are too similar. In the end I will only be paying attention to 4 out of the 7 emote categories (happiness, neutral, sadness, surprise) they do not seem to have any strong relationship as each is strongest with itself, and they have enough entries to be modeled on.



Figure 6: Message lengths all 7 emote labels

To combat our highly skewed data and outliers, we will be filtering our data further according to labeled metrics and using certain threshold cut-offs in terms of deviation, and simply limiting the amount of happiness instances out there. I have also addressed exploring methods to lessen data imbalance but the majority of papers and data science crowd avoid using SMOTE and up sampling techniques. The main issue is that it creates fake data that is not representative of a true sample, and especially dangerous for something as volatile as text data. So with this information, I ran a couple test on baseline models to determine the cut off point for our data limiter. 2,504 appeared to be our sweet spot.

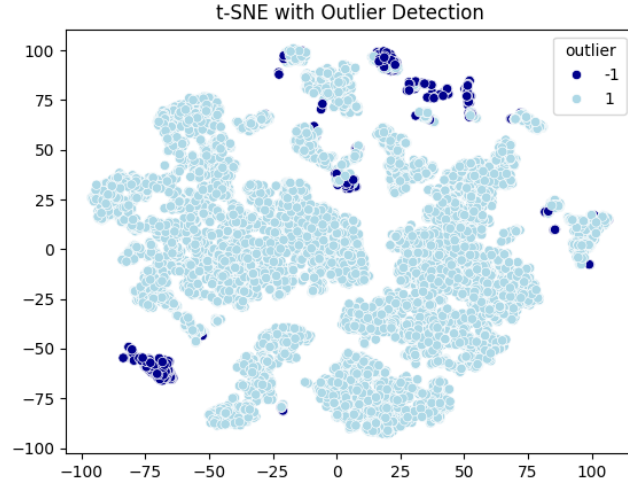


Figure 7: Isolation forest outliers

After fixing the distribution and accounting for the outliers previously mentioned, (message length per emotion, user uniqueness), we arrive at 7,344 messages across 2,934 unique users. Another round of filtering was also done since we were getting unexpected emotes not being aligned with their respective categories. Even though I would have liked to have the data count be higher, I still exceeded our initial proposal data count of 2,500 - 5,000 messages.

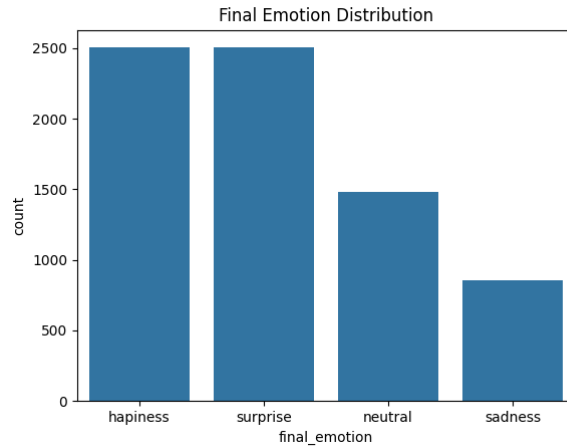


Figure 8: Emotional distribution across 4 emote labels

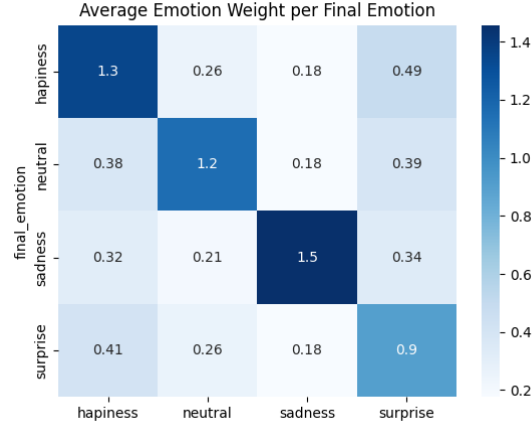


Figure 9: Average emotion per weight across 4 emote labels

Table 3: Top 7 Emotes Per Emotion Category

Emotion	Emote	Count
Happiness	lol	264
Happiness	KEKW	225
Happiness	om	206
Happiness	LUL	199
Happiness	LOL	60
Happiness	classic	59
Happiness	hi	55
Neutral	ok	394
Neutral	YEP	129
Neutral	NODDERS	122
Neutral	zentreTogether	66
Neutral	shyyWave	65
Neutral	Listening	51
Neutral	shyyHug	40
Sadness	o7	230
Sadness	Sadge	173
Sadness	:loudly_crying_face:	65
Sadness	NotLikeThis	58
Sadness	FeelsStrongMan	30
Sadness	limesCry	30
Sadness	HandsUp	19
Surprise	mhm	346
Surprise	ICANT	300
Surprise	Pog	266
Surprise	HYPER	166
Surprise	LETSGO	89
Surprise	GIGACHAD	79
Surprise	AINTNOWAY	66

The last table is a sanity check on the top seven emotes being categorized in tandem with the distribution charts. My only complaint is that sometimes these emotes (not shown on table) fit into more subtle categories that are somewhere between two emotions on the spectrum, duality is something that is harder to capture. Otherwise, the data is good to go for the modeling phase!

Methodology

To review our full pipeline so far and the next steps we will take:

1. **Data Scraping:** Initial data parsing from raw responses and minor text filtering
2. **Data Cleaning I:** Calculations of weights, binning emotions, initial text filtering
3. **Data Cleaning II:** Outliers and distribution fixes, final text filtering and edge-cases
4. **Modeling I:** TF-IDF - Logistic regression, random forest, xg-boost, support vector machines
5. **Modeling II:** Naive Bayes
6. **Modeling III:** DistilBERT-LoRa and FastText
7. **Conclusion:** Ending Notes

Modeling

TF-IDF

As mentioned in the proposal we will be going over a total of 6 models. I threw in an extra one for fun as well (FastText). The first 4 models are TF-IDF (Term Frequency-Inverse Document Frequency), a method that transforms raw informal messages into numerical features that machine learning models can process easier. It counts the term frequencies (how often a word appears) and the inverse (down scaling ones that don't appear frequently). The corresponding weight returned per word is how relevant it is across that specific "document". This can help especially in keyword extraction, accounting for rare words, and distinguishing between common and important words. This will serve as our common baseline method for traditional algorithms.

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_k f_{k,d}} \quad \text{IDF}(t, D) = \log \left(\frac{N}{1 + |\{d \in D : t \in d\}|} \right)$$

The variants explored will consist of logistic regression, random forest, xg-boost, and svm. Random forest will be used to build decision trees based on word importance thresholds, and we will see the gradient boosted optimization which is a similar approach. SVM was picked so we can see how well it does in separating our TF-IDF vectors into hyperplanes, to see what effect it would have in this type of classification. Logistic regression will provide us with a solid baseline on detecting sentiment in emotion since it handles high dimensional data well and is fast to train. It also gives us with a way to deal with sparse data and provides probabilities on how confident the output model is.

Naive Bayes & Bag of Words

Another common method is Naive Bayes in tandem with Bag of Words for text classification tasks. This works well with informal and short messages like chat messages or tweets. In this process each of the words becomes a feature and is represented by a word count vector array.

If you want to think about it, TF-IDF is similar to Bag of Words in that both transform text into numerical representations, but the key difference is how they handle word frequency. While Bag of Words simply counts the occurrences of words in a document, TF-IDF adjusts these counts by considering the frequency of a word within the entire group, giving more importance to terms that are rare across documents but frequent within a specific document.

Naive Bayes on the other hand is a probabilistic classifier and will calculate the probability that a message or text will belong to a certain category based on the probability priors.

$$\hat{c} = \arg \max_{c \in C} \left[P(c) \prod_{i=1}^n P(w_i | c) \right]$$

$$P(c | d) \propto P(c) \prod_{i=1}^n P(w_i | c)$$

In the example equation given, $p(c | d)$ represents the posterior probability, with $p(c)$ being our prior probability of class c , and the $w(i)$ term being the likelihood of word given class.

DistilBERT LoRa and Fasttext

Lastly we will be exploring a more bleeding-edge method which is training on top of a frozen LLM like DistilBERT with a technique called low rank adaptation (LoRa). DistilBERT is a smaller and faster version of it's parent model BERT (Bidirectional Encoder Representations from Transformers). To summarize shortly: it is a type of model that understands language via reading it bidirectionally, looking at the current word and the word after it to help it

understand context.

The transformer within the model takes an input text like any other method, converting our word tokens into numerical vectors that also capture positioning. Then they apply a self attention mechanism that computes each words relevance to other words, giving the model an understanding of context. Afterwards, these scores are fed through multiple feed forward networks and layer normalizations, to gain deeper understanding. Finally depending on the task it will generate the result based on either probability distribution over vocabularies, an input for a classifier, or a sequence of tokens. This can be used for a variety of purposes.

Since this model already has a general learned language pattern, we can continue to train on it further for a specific dataset or task. Fine tuning works as the following: It will pass in our labeled message/emotion classification data, and it will make predictions. The model will correct itself using a loss function and use back propagation to update the weights via the layers that exist (embedding, attention, feed forward, classification).

Initially I did fine tune the model in a traditional sense, where all of the model weights are updated. However, it was not giving me very good results. I believe these poor outcomes were due to my small dataset excessively altering the weights, which may not have been fully representative of the data distribution in the base model. So instead of doing this, I searched for different methods and landed on low rank adaption.

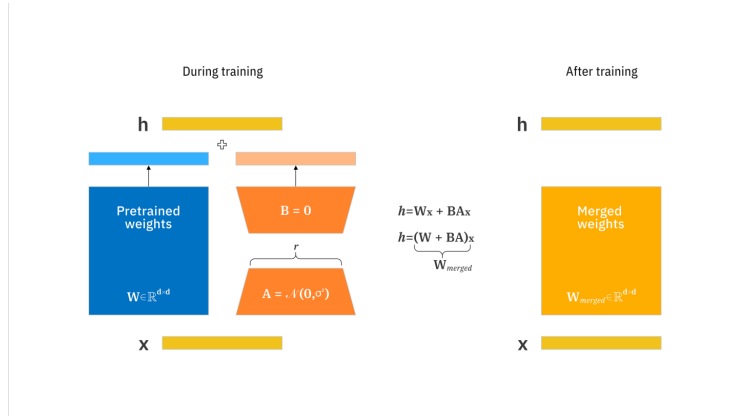


Figure 10: Example LoRa Demonstration from IBM Article

Instead of traditional fine-tuning which modifies and updates the weights for everything in the model, we can use low rank adaption or (LoRa) for short to freeze the weights and add light weight trainable layers on top. This works by inserting low rank matrices into certain parts of the model, which allows for task specific learning without messing around with the pre-trained parameters.

The original model stays intact and only a small number of parameters are optimized. This is a much more efficient, non-destructive, and compute friendly method, while working better with smaller quantities of text.

In addition to all of this, I have thrown another method out there called FastText developed by Facebook. It is a shallow, fast and simple text classifier that works by averaging word vectors to represent a whole sentence or group of words. It considers sub words, because the process breaks down singular tokens/words into character n-grams. The words are then represented by the sum of these n-gram embeddings.

The results are then passed into a classification algorithm like linear or logistic regression. We will be using most default parameters and negative sampling. Even though we already have something similar with TF-IDF, I wanted to see how such an implementation would stack up against all the other ones being tested.

Evaluation and Results

We will be using the standard F1-Score as our main evaluation metric and precision/recall to have some comparison on the side. Despite all the data wrangling and manipulation we did earlier, the data can still be deemed as imbalanced since sadness and neutral have about half the amount as surprise and happiness. In this case it would make the most sense to use F1 score to achieve a balance between precision and recall.

In tandem, Precision and recall tables are shown to assess how well it would perform assuming if the data was balanced. There is no inherent risk to having false positives or making mistakes in our use-case so we can compare these against each other to get a good feel for certain situations.

All TD-IDF, Naive Bayes, and FastText parameters have been tuned to encompass general usage from examples I have seen (multiple sources in classifying text). This includes `n_estimators`, `max_depths`, learning rates, min sample split, n-gram ranges. Naturally, these will vary depending on the algorithm chosen and fed through grid search to find the most optimal solution.

Personally, I am not as experienced with training LoRa for LLMs so I went with a general recommended strategy and tuned the weight decay=0.01, learning rate=3e-5, warm up steps=100, epochs=6, and low rank configurations as I went along. Generally I stayed safe, but venturing just enough out of the comfort zone to explore. This was easily the most time consuming part of the whole project (tuning and running).

On the page below I have outputted an aggregation of all results over the methods I have introduced above in their separate tables, F1 score, precision, recall, and support.

	happiness	neutral	sadness	surprise	macro avg	weighted avg
TF-IDF: Logistic Regression	0.346	0.345	0.305	0.441	0.359	0.373
TF-IDF: Random Forest	0.335	0.298	0.179	0.472	0.321	0.356
TF-IDF: XG-Boost	0.366	0.154	0.114	0.478	0.278	0.332
TD-IDF: SVM	0.440	0.243	0.129	0.435	0.312	0.362
Naive + BoW	0.493	0.180	0.066	0.381	0.280	0.342
DistilBERT-LoRa	0.431	0.324	0.250	0.434	0.360	0.388
Fast Text	0.427	0.267	0.192	0.457	0.336	0.373

Table 4: F1 Across Models

	happiness	neutral	sadness	surprise	macro avg	weighted avg
TF-IDF: Logistic Regression	0.430	0.332	0.257	0.417	0.359	0.386
TF-IDF: Random Forest	0.410	0.303	0.267	0.393	0.344	0.366
TF-IDF: XG-Boost	0.379	0.418	0.500	0.370	0.417	0.398
TD-IDF: SVM	0.380	0.376	0.419	0.396	0.393	0.389
Naive + BoW	0.377	0.420	0.545	0.411	0.438	0.417
DistilBERT-LoRa	0.430	0.326	0.321	0.409	0.372	0.388
Fast Text	0.413	0.327	0.264	0.404	0.352	0.373

Table 5: Precision Across Models

	happiness	neutral	sadness	surprise	macro avg	weighted avg
TF-IDF: Logistic Regression	0.289	0.358	0.374	0.469	0.373	0.374
TF-IDF: Random Forest	0.283	0.294	0.135	0.589	0.325	0.372
TF-IDF: XG-Boost	0.353	0.095	0.064	0.675	0.297	0.377
TD-IDF: SVM	0.521	0.179	0.076	0.483	0.315	0.387
Naive + BoW	0.711	0.115	0.035	0.355	0.304	0.391
DistilBERT-LoRa	0.433	0.323	0.205	0.463	0.356	0.392
Fast Text	0.442	0.225	0.151	0.527	0.336	0.386

Table 6: Recall Across Models

	happiness	neutral	sadness	surprise	macro avg	weighted avg
TF-IDF: Logistic Regression	501	296	171	501	1469	1469
TF-IDF: Random Forest	501	296	171	501	1469	1469
TF-IDF: XG-Boost	501	296	171	501	1469	1469
TD-IDF: SVM	501	296	171	501	1469	1469
Naive + BoW	501	296	171	501	1469	1469
DistilBERT-LoRa	501	310	176	482	1469	1469
Fast Text	500	315	185	469	1469	1469

Table 7: Support Across Models

Here is the generated loss curve as I was training the LoRa. In terms of performance out of all the combinations of my runs, the results usually peak in terms of F1 around 4-5 epochs, which I have deemed a good place to stop.

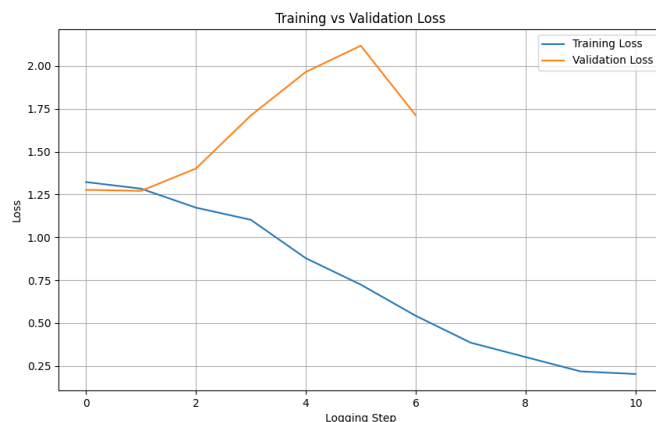


Figure 11: Training and Validation Loss on LoRa

Overall, to explain what is going on in this graph, while the training loss quickly drops, validation loss increases. This is a tell-tale sign that the model is overfitting, and this is not a shock at all. Like mentioned before it requires a lot more data and the results are showing that the model is simply memorizing versus learning to generalize. Another future option to consider is trying a smaller model or simply gathering a lot more data (50,000+) entries. However, these results are valid given the time and resources I had.

Out of all the F1 scores here we can see that DistilBERT-LoRa performs the best, clocking in with the highest weighted averages and macro average (0.388, 0.360) respectively. Most may say that these scores are quite abysmal but, given that LoRas should be trained on more than just 7,000 instances of data. Despite limited data, DistilBERT-LoRa outperformed all other models, highlighting its robustness. That's just the power of transformers and low rank adaptation at play. Given more data, DistilBERT-LoRa could reasonably be expected to double its performance.

Naive bayes and bag of words performs surprisingly poor as fast text and TF-IDF logistic regression comes in second. This may be due to bag of words treating words as discrete tokens and losing the context. In our case it seems like TF-IDF weights are helping with our scenario boosting performance by a little bit. While all the other models perform similarly and slightly worse than the ones mentioned, RF, xg-boost, and SVMs in theory do not handle sparse high-dimension features (short-text) well. I was curious to see how these would

perform, not necessarily that I thought they were the best tools for the job.

The overall performance of these models are not very good, if you like to think from the perspective of 1/4 being a random guess. Disregarding this notion, I would not say they are useless. Being 0.10 better than a guess is still notable given the training circumstances. DistilBERT-LoRa has yet to see its potential, and like I keep mentioning, it will require much more data than the project scope suggested.

Below are the confusion matrix results for your perusal:

hapiness	neutral	sadness	surprise
145	98	79	179
54	106	39	97
33	21	64	53
105	94	67	235

Table 8: TF-IDF: L-Regression

hapiness	neutral	sadness	surprise
162	78	37	224
75	73	16	132
41	15	39	76
112	70	30	289

Table 9: TF-IDF: Random Forest

hapiness	neutral	sadness	surprise
177	23	4	297
93	28	3	172
50	4	11	106
147	12	4	338

Table 10: TF-IDF: xg-boost

hapiness	neutral	sadness	surprise
261	44	7	189
124	53	5	114
86	6	13	66
215	38	6	242

Table 11: TF-IDF: SVM

hapiness	neutral	sadness	surprise
356	20	1	124
183	34	1	78
104	8	6	53
301	19	3	178

Table 12: Naive Bayes + BoW

hapiness	neutral	sadness	surprise
239	63	30	143
114	103	24	89
71	11	30	37
199	74	27	215

Table 13: DistilBERT LoRa

hapiness	neutral	sadness	surprise
219	59	33	189
96	73	22	124
67	25	30	63
139	55	24	251

Table 14: FastText

Conclusion

Even though the overall the performance of all the methods tested TF-IDF: LR, RF, XG, SVM, Naive Bayes + BoW, DistilBERT LoRa, and FastText, yielded sub par results. I got a chance to test out a wide variety of approaches to tackle a multi-classification problem. The most interesting one was building a low rank adaptation on top of a large language model like DistilBERT. The idea was fresh to me, but I realized that the scope of my project held me back from achieving good results.

Unfortunately models like that require a lot of data. Given more a better time-frame, I would be able to aggregate more. Like mentioned previously, certain emotions are harder to capture and will take time, but given enough patience it can be done, in tandem with better data cleaning. However, I was surprised it still performed the best out of all the models tested.

Given a future approach, I would like to see if incorporating some of what the content creator is saying as context via speech to text would aid in labeling. This

would allow for a more accurate interpretation and perhaps a more definitive labeling method that was not so emotive reliant. This is a rabbit hole I will save for another day, thanks for reading!